

Claude Code 推出 /ultrareview 超级审查功能，20 美金一次，10 分钟干完

原创 John AGI Hunt 2026年4月23日 10:47 浙江



AGI Hunt

关注AGI的沿途风景！前网易资深技术专家；出海某垂类 Agent 方向创业中；佛系写作
1815篇原创内容

>

公众号

今天介绍 Claude Code 上线的一个新功能：`/ultrareview`。

一句话概括：它会在云端同时派出多个 AI 审查员，帮你在合并代码之前把 Bug 揪出来。



这个功能其实在上周 Claude Opus 4.7 发布时就提到了，当时 Anthropic 在发布公告里写的是：

“新的 `/ultrareview` 斜杠命令会创建一个专门的代码审查会话，通读你的改动，标记出一个认真的代码审查者会注意到的 Bug 和设计问题。”



ultrareview 演示动图

根据 Claude Code 的更新日志，`/ultrareview` 在 4 月 17 日的 v2.1.113 版本中首次加入，4 月 20 日的 v2.1.116 又做了一轮优化（启动速度更快、确认框里新增了 diffstat 统计）。



ClaudeDevs  
@ClaudeDevs



Hunt



 显示翻译

New in Claude Code: `/ultrareview` (research preview) runs a fleet of bug-hunting agents in the cloud.

Findings land in the CLI or Desktop automatically. Run it before merging critical changes—auth, data migrations, etc.

Pro and Max users get 3 free reviews through 5/5.  公众号 · AGI Hunt

今天，Anthropic 官方开发者账号 @ClaudeDevs 正式宣布了这个功能。

01

不止一个审查员

`/ultrareview` 和普通的 `/review` 最大的区别在于，它不是一个 Agent 在看你的代码。

它是一群。

当你在 CLI 里敲下 `/ultrareview`，Claude Code 会把你的代码仓库打包上传到云端沙箱，然后在那里启动一整支 reviewer Agent 编队，**并行地**审查你的改动。



多 Agent 工作流程

每个 Agent 独立工作，找到可疑的地方之后，还会交给另一个 Agent **独立验证**。

只有经过验证确认的 Bug，才会最终反馈给你。

这就像是请了一组代码审查员，每人看完之后还要交叉检查，确保不是误报。

这种「多 Agent + 独立验证」的模式带来了几个好处：

- **信噪比高**：报出来的基本都是真问题，不是代码风格建议

- **覆盖面广**：多个 Agent 并行探索，能发现单次审查容易遗漏的交叉问题
- **不占本地资源**：整个过程在云端跑，你的终端该干嘛干嘛

02

3+1 架构

根据 MindStudio 对其架构的分析，ultrareview 的编队其实并不大，但分工很精妙：**3 个 Explorer Agent + 1 个 Critic Agent**。

3 个 Explorer 各自拿到一份代码改动，独立分析。

它们各有独立的 context window，互不干扰，可能一个盯安全漏洞，一个看逻辑错误，一个查性能隐患。



然后，1 个 Critic Agent 负责「质检」。

它拿到三个 Explorer 的全部产出，逐条检查：这个发现能不能复现？是不是只看了片面？有没有遗漏的场景？

只有经过 Critic 验证的 Bug，才会出现在最终报告里。

这套机制解释了为什么 ultrareview 的误报率能比单次 `/review` 低那么多，毕竟三个人各自找问题，再让第四个人专门挑毛病，假阳性能活下来才怪。

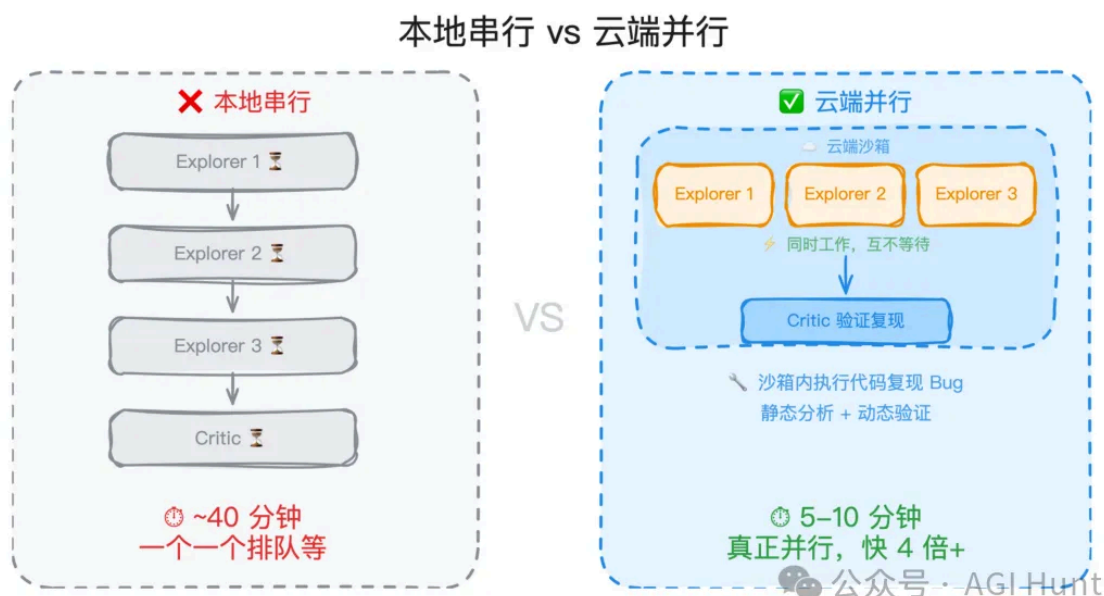
03

为什么在云端

你应该和我一样，有一个问题：为什么不能在本地跑呢？

答案其实不复杂。ultrareview 需要同时启动 4 个 Agent，每个都要独立的模型调用和 context window。在本地跑，意味着你的终端要串行处理 4 轮完整的模型交互，光等待时间也就够呛了。

而在云端沙箱里，这 4 个 Agent 可以**真正并行**，各自同时工作，最后汇总。



本地串行 vs 云端并行

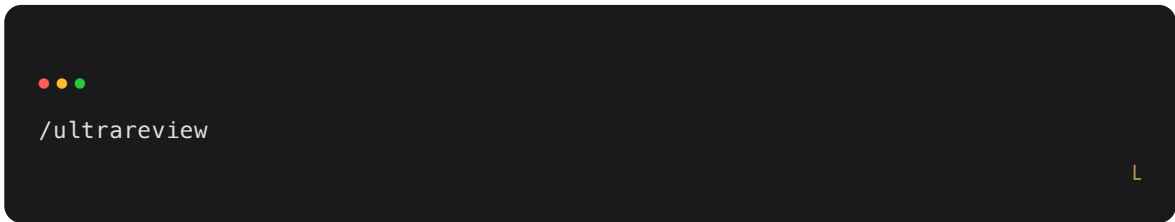
还有一点是：Critic Agent 在验证 Bug 时，需要在沙箱环境里实际执行代码来复现问题，不只是做静态分析。这也是本地环境不容易做到的。

这也是为什么 ultrareview 必须跑在云端，而不是你自己的电脑上。

不过可能还有另一个原因是，**这样更好收钱啊！**以及，云端原因，所以用的没准是内部更强的 Mythos 模型？

实际操作

用法很简单。在任何 Git 仓库里，直接输入：

A terminal window with a dark background. At the top left, there are three colored dots (red, yellow, green). The text `/ultrareview` is entered. On the right side, there is a small yellow icon that looks like a stylized 'L' or a logo.

```
/ultrareview
```

不加参数的话，它会审查你当前分支和默认分支之间的 diff，包括暂存和未暂存的改动。如果你想审查一个 GitHub PR，就加上 PR 号：

A terminal window with a dark background. At the top left, there are three colored dots (red, yellow, green). The text `/ultrareview 1234` is entered. On the right side, there is a small yellow icon that looks like a stylized 'L' or a logo.

```
/ultrareview 1234
```

PR 模式下，云端沙箱会直接从 GitHub 拉取 PR 内容，不需要上传本地代码。

启动前，Claude Code 会弹出确认框，告诉你审查范围、剩余免费次数和预估费用。从下面这张官方演示截图能看到：审查 `feat/auth` 分支对 `main` 的改动，12 个文件、340 行插入、89 行删除，确认后就开始了。



公众号 · AGI Hunt

确认审查范围

确认之后，审查就在后台跑了。一般需要 5 到 10 分钟。

然后你就可以去干别的了。

终端底部会显示 ultrareview 的进度状态，从 `setting up` 到 `12 found · 1 verified · 0 refuted`，你随时都能瞄一眼进度。



后台运行中

审查完成后，结果会以通知的形式出现在你的 CLI 或 Desktop 里。

来看看这个演示里的结果：ultrareview 总共发现了 4 个已验证问题，8 个被判定为误报直接过滤掉了。每个发现都标明了具体的文件位置和问题解释：



审查结果

比如第一个问题是 `src/auth/session.ts:142` 的竞态条件，token 刷新可能在 `destroy()` 之后触发，写入已释放的 `handle`。第二个是 `src/net/fetchWithRetry.ts:58` 的无限重试循环，服务器返回 429 时没有最大重试次数限制。

这些都是单次 review 容易漏掉、但上了线就会出事的问题。

用 `/tasks` 可以随时查看正在跑的审查进度，也可以中途叫停。

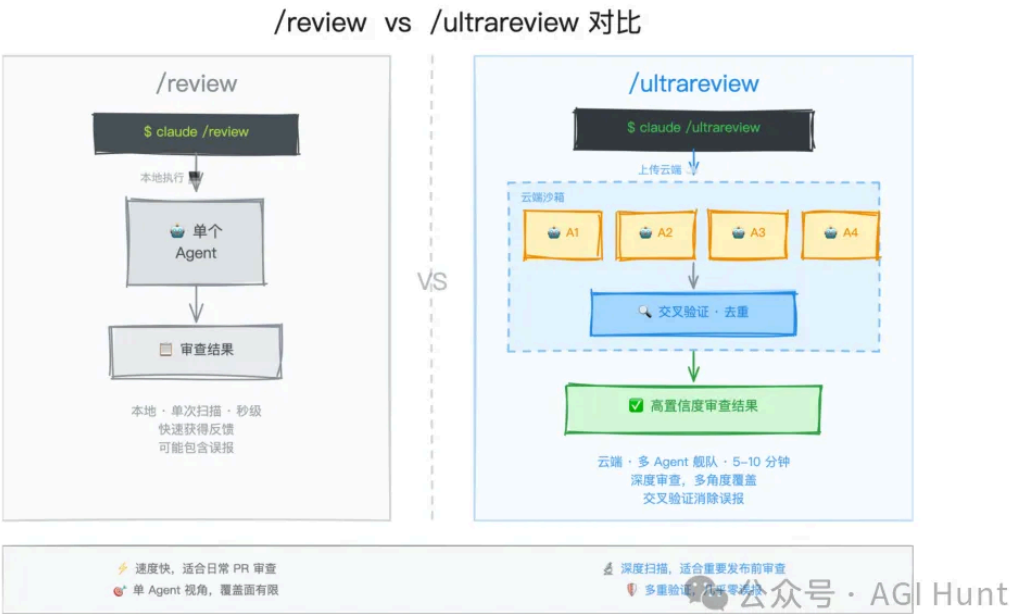
05

怎么选

`/review` 和 `/ultrareview` 并不冲突，适用场景不同。

	/review	/ultrareview
运行位置	本地	云端沙箱
审查深度	单次扫描	多 Agent + 交叉验证
耗时	几秒到几分钟	5 到 10 分钟
费用	算在日常用量里	免费额度用完后 \$5-\$20/次
适合场景	写代码时随手查	合并前的最后一道关

简单说，`/review` 是你写代码时的随手检查，`/ultrareview` 是 PR 合并前的深度体检。



对比

认证变更、数据迁移、权限逻辑这类出了问题代价很大的改动，应该就是 `/ultrareview` 的主场。

06

多少钱

Pro 和 Max 用户各有 **3 次免费体验**，有效期到 2026 年 5 月 5 日。Team 和 Enterprise 用户没有免费额度。

免费次数用完之后，每次审查大约 **\$5 到 \$20**，取决于改动的规模。费用从 extra usage 里扣，不占你的计划用量。



公众号 · AGI Hunt

定价

也就是说，你得先在账户里开启 extra usage 才能用付费的 ultrareview。没开的话，Claude Code 会拦住你，并给你一个跳转链接去打开。

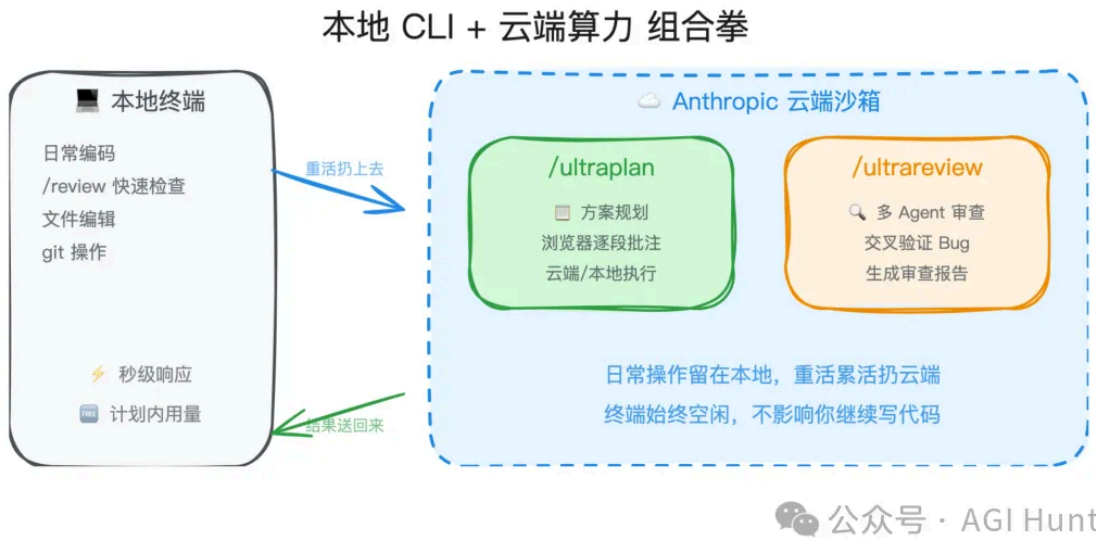
\$5 到 \$20 一次审查.....虽然确实挺贵，但换个角度想，一个线上 Bug 的修复成本，应该远不止这个数。

07

云端策划

顺便提一嘴，和 /ultrareview 一起上线的还有一个 `/ultraplan`。

如果说 ultrareview 是合并前的「深度体检」，那 ultraplan 就是动手前的「云端策划」。你在 CLI 里输入 `/ultraplan` 把认证服务从 session 迁移到 JWT，它会在云端启动一个 Claude Code on the web 会话，帮你做方案规划。



本地 + 云端组合

规划好之后，你可以在浏览器里逐段批注、要求修改，满意了再选择.....是在云端直接执行，还是发回本地终端。

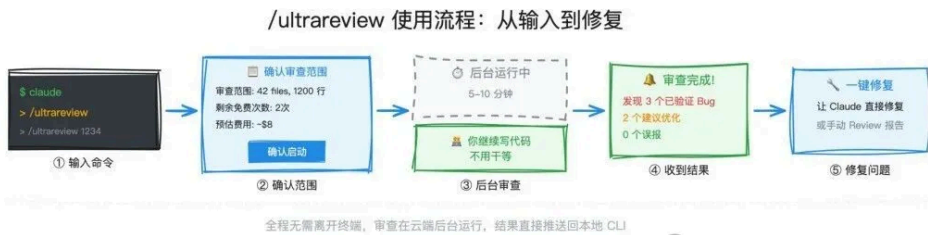
Anthropic 这是在搭建一套「本地 CLI + 云端算力」的组合拳：**日常操作在本地，重活累活扔云端。**

08

限制条件

本质上，ultrareview 依赖的是 Claude Code on the web 的云端基础设施。所以它的可用范围，和 Claude Code on the web 基本一致。

因此，用之前有几个点需要注意：



使用流程总览

开启了零数据保留（Zero Data Retention）的组织也用不了。如果仓库太大传不上去的话，Claude Code 会提示你用 PR 模式。

另外，需要用 Claude.ai 账号登录，API key 不行（需要先用订阅账号 /login）。所以也不支持 Amazon Bedrock、Google Cloud Vertex AI 和 Microsoft Foundry 等渠道。

什么？中转站？

显然不能。



相关链接：

- ultrareview 文档：<https://code.claude.com/docs/en/ultrareview>
- ultraplan 文档：<https://code.claude.com/docs/en/ultraplan>
- Claude Code：<https://claude.com/claude-code>

👍 喜欢作者